

编译器设计实验-第四次实验

计算机 2303 尤颢辰

2026 年 5 月 26 日

1 实验任务（必做）

1.1 任务描述

1. 在 Lab3 基础上增加 SLR(1) 需要的数据结构;
2. 计算 FIRST 集;
3. 计算 FOLLOW 集;
4. 构造 ACTION 表（移进/归约/接受，FOLLOW 集限定归约）;
5. 构造 GOTO 表;
6. 检测 SLR(1) 冲突并输出;
7. 输出完整分析表。

1.2 代码框架

我们整体的代码在 Lab3 里实现的 LR(0) 文法的基础之上实现。所以下面列出新增的数据结构以及函数：

表 1: 数据结构表

数据结构名	含义
Action	表示一个 ACTION 表项，type 区分移进、归约、接受
SLR1Table	完整的 SLR(1) 分析表，包含 ACTION、GOTO 及冲突列表
SLR1Result	顶层结果，聚合文法、项目集、转移、FIRST/FOLLOW 和分析表

表 2: 函数表

函数名	作用
compute_first()	计算每个非终结符的 FIRST 集
compute_follow()	计算每个非终结符的 FOLLOW 集
build_action_table()	构造 ACTION 表
build_goto_table()	构造 GOTO 表
build_slr1_table()	组装上面两张表，确定行列表头
build_slr1()	串联所有步骤

下面来实现这些函数。

1.3 Task1: 计算 FIRST 集

FIRST 集代表从一个符号出发，所有可能推导出的串的第一个终结符的集合。计算方法分两部分：首先，终结符的 FIRST 集就是它本身。

其次，对非终结符，遍历所有产生式 $A \rightarrow Y$ ，把 $FIRST(Y)$ 并入 $FIRST(A)$ 。由于本实验文法不含 ε 产生式，所以右部只需看第一个符号即可。重复这个过程直到所有的 FIRST 集不发生变化。关键实现如下：

```
1 bool changed = true;
2 while (changed)
3 {
4     changed = false;
5     for (int i = 0; i < (int)grammar.productions.size(); i++)
6     {
7         const string &A = grammar.productions[i].left;
8         const string &Y1 = grammar.productions[i].right[0];
9         for (set<string>::iterator it = first[Y1].begin(); it != first[Y1].end(); ++it)
10             if (first[A].insert(*it).second)
11                 changed = true;
12     }
13 }
```

1.4 Task2: 计算 FOLLOW 集

FOLLOW(A) 表示所有句型中紧跟在非终结符 A 后面的终结符集合 (含输入结束符)。只有当前输入符号属于 FOLLOW(A)，才能把 A 对应的产生式归约。

FOLLOW(A) 的计算实现如下：

首先把输入结束符加入原文法开始符号 E 的 FOLLOW 集；

对每条产生式 $B \rightarrow \alpha A \beta$ ，遍历其右部所有非终结符 A，若 β 非空，将 $FIRST(\beta$ 的第一个符号) 加入 FOLLOW(A)，否则将 FOLLOW(B) 并入 FOLLOW(A)。不断重复，直到所有 FOLLOW 集不再发生变化。

关键实现如下：

```
1 follow[grammar.start_symbol].insert("$");
2 bool changed = true;
3 while (changed)
4 {
5     changed = false;
6     for (int i = 0; i < (int)grammar.productions.size(); i++)
7     {
8         const string &A = grammar.productions[i].left;
9         const vector<string> &rhs = grammar.productions[i].right;
10        for (int j = 0; j < (int)rhs.size(); j++)
11        {
12            const string &B = rhs[j];
13            if (grammar.nonterminals.count(B) == 0) continue;
14            if (j + 1 < (int)rhs.size())
15            {
16                const set<string> &firstBeta = first.at(rhs[j + 1]);
17                for (set<string>::iterator it = firstBeta.begin(); it != firstBeta.end(); ++it)
18                    if (follow[B].insert(*it).second) changed = true;
19            }
20        }
21    }
22 }
```

```

20         else
21         {
22             for (set<string>::iterator it = follow[A].begin(); it != follow[A].end(); ++it)
23                 if (follow[B].insert(*it).second) changed = true;
24         }
25     }
26 }
27 }

```

1.5 Task3: 构造 ACTION 表

ACTION 表决定在每个状态下遇到某个终结符时该做什么。对项目集 I_i 中的每个项目，分三种情况：

情况 A: 移进, 若项目形如 $A \rightarrow \alpha a \beta$, 在转移表里找 $I_i \rightarrow a \rightarrow I_j$, 则填 $ACTION[i][a] = sj$;

情况 B: 归约, 若项目形如 $A \rightarrow \alpha \cdot$, 设产生式编号为 k , 对 $\forall a \in FOLLOW(A)$ 填 $ACTION[i][a] = rk$ 。

情况 C: 接受, 若项目是 $E' \rightarrow E \cdot$, 填 $ACTION[i][\$] = acc$ 。

遇到同一格要填两个不同动作时, 记录冲突信息 (移进-归约或归约-归约) 但保留先写入的值。

关键的实现如下:

```

1  if (item.dot_pos < (int)prod.right.size())
2  {
3      const string &a = prod.right[item.dot_pos];
4      if (grammar.terminals.count(a) == 0) continue;
5      for (int k = 0; k < (int)transitions.size(); k++)
6      {
7          if (transitions[k].from != i || transitions[k].symbol != a) continue;
8          Action shift; shift.type = 1; shift.value = transitions[k].to;
9          if (action[i].count(a) == 0) action[i][a] = shift;
10         else if (action[i][a].type != shift.type)
11             conflicts.push_back("I" + to_string(i) + " 在符号 " + a + " 上存在移进-归约冲突");
12         break;
13     }
14 }
15 else if (item.production_index == 0)
16 {
17     Action acc; acc.type = 3;
18     if (action[i].count("$") == 0) action[i]["$"] = acc;
19 }
20 else
21 {
22     const set<string> &fa = follow.at(prod.left);
23     for (set<string>::iterator it = fa.begin(); it != fa.end(); ++it)
24     {
25         Action reduce; reduce.type = 2; reduce.value = item.production_index;
26         if (action[i].count(*it) == 0) action[i][*it] = reduce;
27         else /* 记录冲突 */;
28     }
29 }

```

1.6 Task4: 构造 GOTO 表

GOTO 表处理归约后的状态跳转。即用某条产生式归约出非终结符 X 后，应该从当前栈顶状态跳到哪里。其实现最简单，直接遍历已有的转移表，把标号是非终结符的那些转移拿出来填进表即可。关键实现：

```
1 int n = (int)item_sets.size();
2 vector<map<string, int>> go_to(n);
3 for (int i = 0; i < (int)transitions.size(); i++)
4 {
5     const string &X = transitions[i].symbol;
6     if (grammar.nonterminals.count(X) == 0) continue;
7     go_to[transitions[i].from][X] = transitions[i].to;
8 }
```

1.7 输出结果

由于云服务器额度耗尽，于是在本地运行。使用 and PPT 中类似的方法临时修改提示符，结果如下：

```
PS E:\2026_Junior_S2\CompilerLab> function prompt { "[尤颖辰@本地 $(Split-Path -Leaf (Get-Location))]\$ " }
>>

FIRST 集
FIRST(): (
FIRST()): )
FIRST(*): *
FIRST(+): +
FIRST(E): ( id
FIRST(E'): ( id
FIRST(F): ( id
FIRST(T): ( id
FIRST(id): id

FOLLOW 集
FOLLOW(E): $ ) +
FOLLOW(E'):
FOLLOW(F): $ ) * +
FOLLOW(T): $ ) * +

SLR(1) ACTION 表
状态 ( ) * + id $
0 s4 s5 acc
1 r4 r4 r4 r4
2 r2 s7 r2 r2
3 s4 s5 r6
4 s4 r6 r6 r6 s5 r6
5 s4 s5
6 s11 s6
7 r1 s7 r1 r1
8 r3 r3 r3 r3
9 r5 r5 r5 r5
10
11

SLR(1) GOTO 表
状态 E F T
0 1 2 3
1
2
3
4 8 2 3
5
6 2 9
7 10
8
9
10
11

SLR(1) 冲突检查
未发现冲突，该文法是 SLR(1) 文法

[尤颖辰@本地 Lab4]\$
```

图 1: Result

2 实验任务（选做）

2.1 任务描述

1. 在之前完成的可视化框架的基础之上，更新 Lab4 的功能；
2. 使用 flex 作为词法分析工具，使用 bison 作为语法分析工具，感受一下自动化工具。

2.2 成果展示

完成的网页端可视化展示如下（Repo: <https://github.com/Yukino-chyan/CompilerWeb>）:

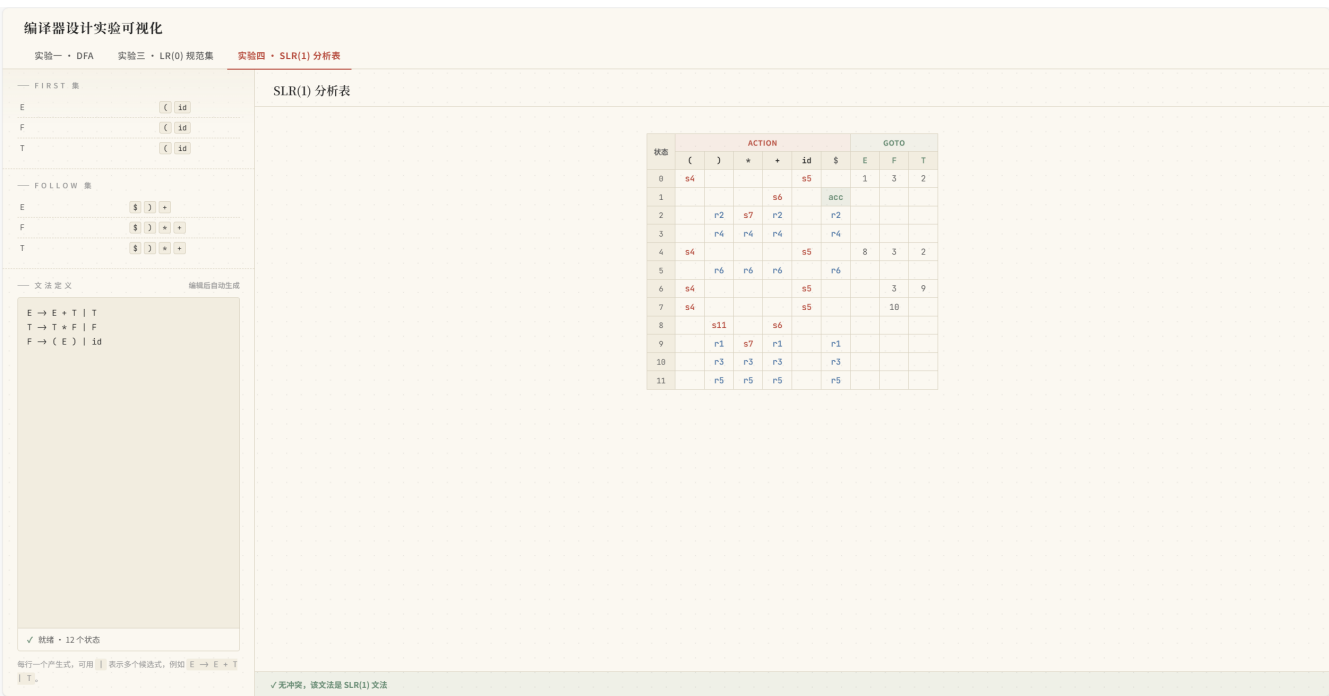


图 2: 可视化

更新可视化网页的功能后，我们体验一下自动化工具。下载安装包到本地，解压后在 powershell 把它加进 PATH:

```
PS E:\2026_Junior_S2\CompilerLab> $env:PATH += ";E:\2026_Junior_S2\CompilerLab\flex_bison"
PS E:\2026_Junior_S2\CompilerLab> win_flex --version
win_flex.exe 2.6.4
PS E:\2026_Junior_S2\CompilerLab> win_bison --version
bison (GNU Bison) 3.8.2
Written by Robert Corbett and Richard Stallman.

Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

图 3: 自动化工具

之后我们编写两个规则文件，调用 win_flex 与 win_bison 工具分别生成词法分析器与语法分析器。比如使用 flex 生成词法分析器，我写的规则文件 expr.l 核心部分如下，其等价于 Lab1/2 手工实现的词法分析：

```

1 %%
2 [a-zA-Z_][a-zA-Z0-9_]* { return ID; }
3 [0-9]+ { return ID; }
4 "+" { return '+'; }
5 "*" { return '*'; }
6 "(" { return '('; }
7 ")" { return ')'; }
8 [ \t\r\n] { /* 跳过空白 */ }
9 . { printf("未知字符: %s\n", yytext); }
10 %%

```

语法分析器的规则文件 expr.y 核心部分如下：

```

1 %%
2 E : E '+' T { printf("归约: E -> E + T\n"); }
3   | T       { printf("归约: E -> T\n"); }
4   ;
5
6 T : T '*' F { printf("归约: T -> T * F\n"); }
7   | F       { printf("归约: T -> F\n"); }
8   ;
9
10 F : '(' E ')' { printf("归约: F -> ( E )\n"); }
11   | ID       { printf("归约: F -> id\n"); }
12   ;
13 %%

```

之后我们相应地执行命令，得到自动化工具生成的词法分析器与语法分析器，再把两个生成的 C 文件一起编译链接成可执行文件：

```

PS E:\2026_Junior_S2\CompilerLab> cd E:\2026_Junior_S2\CompilerLab\flex_bison
PS E:\2026_Junior_S2\CompilerLab\flex_bison> win_bison -d expr.y
PS E:\2026_Junior_S2\CompilerLab\flex_bison> win_flex expr.l
PS E:\2026_Junior_S2\CompilerLab\flex_bison> g++ -o expr.exe expr.tab.c lex.yy.c -static

```

图 4: 得到可执行文件

运行这个可执行文件，看看效果：

```

PS E:\2026_Junior_S2\CompilerLab\flex_bison> cd E:\2026_Junior_S2\CompilerLab\flex_bison
>> win_flex expr.l
>> g++ -o expr.exe expr.tab.c lex.yy.c -static
>> .\expr.exe
>>
请输入表达式 (如 id + id * id)，回车确认:
1+2*5
归约: F -> id
归约: T -> F
归约: E -> T
归约: F -> id
归约: T -> F
归约: F -> id
归约: T -> T * F
归约: E -> E + T
语法分析完成。

```

图 5: Result

可以看到，自动化工具形成的文件已经能够完成归约任务了。的确十分方便。

3 个人总结

本次实验在 Lab3 LR(0) 的基础上完成了 SLR(1) 分析表的自动生成。核心工作是正确求出 FIRST/FOLLOW 集，并以此为依据将归约动作写入 ACTION 表，从而把 LR(0) 的移进-归约冲突消解为 SLR(1) 级别的判断。实验加深了我对“集合不断扩展直到稳定”这一思想的理解，也让我体会到分析表生成的核心逻辑和 SLR(1) 相对于 LR(0) 的改进之处。之后在选做部分，我实现了 SLR(1) 表的网页端可视化，以及体验了自动化工具完成词法分析/语法分析的过程，收获颇丰。